



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Malaysia-Japan
International
Institute of Technology
(MJIT)

SECJ 3483 – WEB TECHNOLOGY

INDIVIDUAL ASSIGNMENT (20%)

Assignment : DHL Incident Reporting System (Web Application + RPA)

Scenario 2

Done by:

Name: Rashadul Nafis Riyad

Matric No: A22MJ3010

Section: 16

SECTION 1 — INTRODUCTION

1.1 Competition Background

This report outlines the creation of an AI-Enhanced Incident Reporting and Resolution System for the DHL Asia Pacific Shared Services Digital Automation Challenge 3.0. This competition, in partnership with Universiti Teknologi Malaysia, aims to address real operational challenges faced by DHL's logistics and finance teams in the Asia Pacific region. Participants utilize Robotic Process Automation (RPA) with UiPath and may incorporate a Large Language Model (LLM). Evaluations focus on technical skills, business logic, and real-world applicability, emphasizing practical solutions for enterprise use.

1.2 Scenario Description and Justification

Scenario 2 — AI-Enhanced Incident Reporting and Resolution System was selected for this submission.

DHL's Customer Support teams receive a high volume of daily incident reports — covering late deliveries, address issues, damaged parcels, system errors, and customer complaints— arriving through multiple inconsistent channels including emails, chat messages, phone call notes, images, and handwritten warehouse notes. Because this information is unstructured and fragmented, DHL struggles to identify issues accurately, assign incidents correctly, prioritise urgently, and report to management reliably. The objective is to design and develop an AI-powered Incident Reporting System that automates the full process of collecting, understanding, assigning, and tracking incident reports — end to end.

1.2.1 Justification for Choosing Scenario 2

Scenario 2 was chosen over Scenario 1 for three specific reasons.

- First, the problem directly maps to all three required components — RPA, web application, and LLM — in a technically meaningful way. The RPA layer ingests and routes raw inputs, the LLM extracts intelligence from unstructured content, and the web application serves as the operational command centre. Each component has a clear, necessary role.
- Second, the scenario involves multi-format input handling — text, PDF, DOCX, and image files. Images of handwritten warehouse notes and damaged parcels require genuine AI vision capabilities, adding a dimension of sophistication that reflects real-world automation challenges.
- Third, the business impact is immediate and measurable. A working solution produces tangible value — faster resolution times, fewer duplicate tickets, correct department routing, and live management reporting — all demonstrable in a system walkthrough.

1.3 The Problem — Unstructured Incidents at Scale

DHL's Customer Support and Logistics Operations teams handle a high volume of incident reports daily — spanning late deliveries, COD disputes, damaged parcels, missing packages, incorrect addresses, system errors, and customer complaints. The core challenge is not the volume but how incidents arrive: simultaneously across multiple disconnected channels — WhatsApp messages, email threads, phone call notes, images, and handwritten warehouse notes — each fragment incomplete and unconnected to the others.

The consequences are severe and systemic:

- **Misidentification:** Incomplete, single-channel inputs lead to incorrect or missing responses.
- **Duplicate tickets:** Separate agents log the same incident independently, creating wasted effort.
- **Incorrect routing:** Without full context at intake, department assignment becomes a manual guess.
- **No prioritisation:** Urgent incidents are not flagged automatically, resulting in SLA breaches.
- **No unified visibility:** Departments work in isolation with no shared view of resolution progress.
- **No reliable reporting:** Manual compilation produces inconsistent and time-consuming management reports.

Every failure flows from one root cause: the absence of structured data at the point of entry.

1.4 The Proposed Solution

The system developed in this project is a fully integrated, AI-powered Incident Reporting System that automates the complete lifecycle of an incident — from the moment a raw, unstructured input arrives from any channel, to the moment a clean, resolved incident is reported to management with full traceability.

The system is built on three tightly integrated layers:

1. **A secure React-based web application** — serving as the operational command centre where agents manage incidents, track SLA progress, in app-notifications for SLA status, view department assignments, monitor dashboards, and generate operational reports.
2. **A UiPath Studio Web RPA automation layer** — running a fully automated processing pipeline triggered every minute, handling input ingestion, content extraction across multiple file formats, and posting structured incident cards to the web application.
3. **An AI intelligence layer** — powered by Anthropic's Claude Haiku and Claude Sonnet models via UiPath GenAI activities, performing language detection, named entity recognition, sentiment analysis, content rewriting, summarisation, semantic duplicate detection, incident classification, and full structured incident card generation. A rule-based fallback mechanism ensures the pipeline continues to function reliably even when AI services are unavailable.

The fundamental design principle of this system is the imposition of structure on chaos at the earliest possible point in the pipeline. Rather than allowing unstructured data to propagate through the organisation, the system intercepts every raw input, understands it semantically, determines the nature of the issue, assesses the severity of the incident, connects related information, classifies it intelligently, and delivers a clean, actionable incident record to the right team — automatically, and typically within 30 seconds of upload.

1.5 Technology Overview

Component	Technology
Frontend	React, Vite, Tailwind CSS
Backend	Node.js, Express.js
Database	PostgreSQL (Supabase)
Authentication	JSON Web Tokens (JWT)
Deployment	Render (backend), local (frontend)
RPA	UiPath Studio Web (cloud robot)
Email	Gmail via UiPath connector
AI Models	Claude Haiku (NLP tasks), Claude Sonnet (reasoning & generation), GPT (image analysis)Gmail via UiPath connector
API	RESTful API

1.6 Report Structure

The remainder of this report is organised as follows:

- **Section 2 — System Architecture:** Three-layer design, data flow, and key architectural decisions.
- **Section 3 — API and JSON Structure:** REST API endpoints, incident card JSON format, and RPA-to-web integration.
- **Section 4 — User Interface Screenshots:** Annotated screenshots of the key web application pages.
- **Section 5 — RPA Workflow and Automation Explanation:** UiPath pipeline stages, AI processing, fallback mechanism, and error handling.

SECTION 2 — SYSTEM ARCHITECTURE

2.1 Architecture Overview

As shown in Figure 1, the DHL Incident Management System is built on a four-layer architecture: a React-based frontend for user interaction, a Node.js backend that enforces all business logic, a PostgreSQL database for persistent structured storage, and a UiPath Studio Web RPA layer that operates as an independent automated pipeline. Each layer has a clearly defined and non-overlapping responsibility. Together, they form a system where raw, unstructured incident reports arrive at one end and clean, trackable, SLA-monitored incident records emerge at the other — without any manual intervention.

Communication between the frontend and backend is handled exclusively via HTTPS REST API calls returning JSON responses. The RPA layer interacts with the backend through the same REST API, treating the backend as its single point of integration. The database is never accessed directly from the frontend or the RPA layer — all data operations pass through the backend, which enforces validation, authorisation, and business rules consistently.

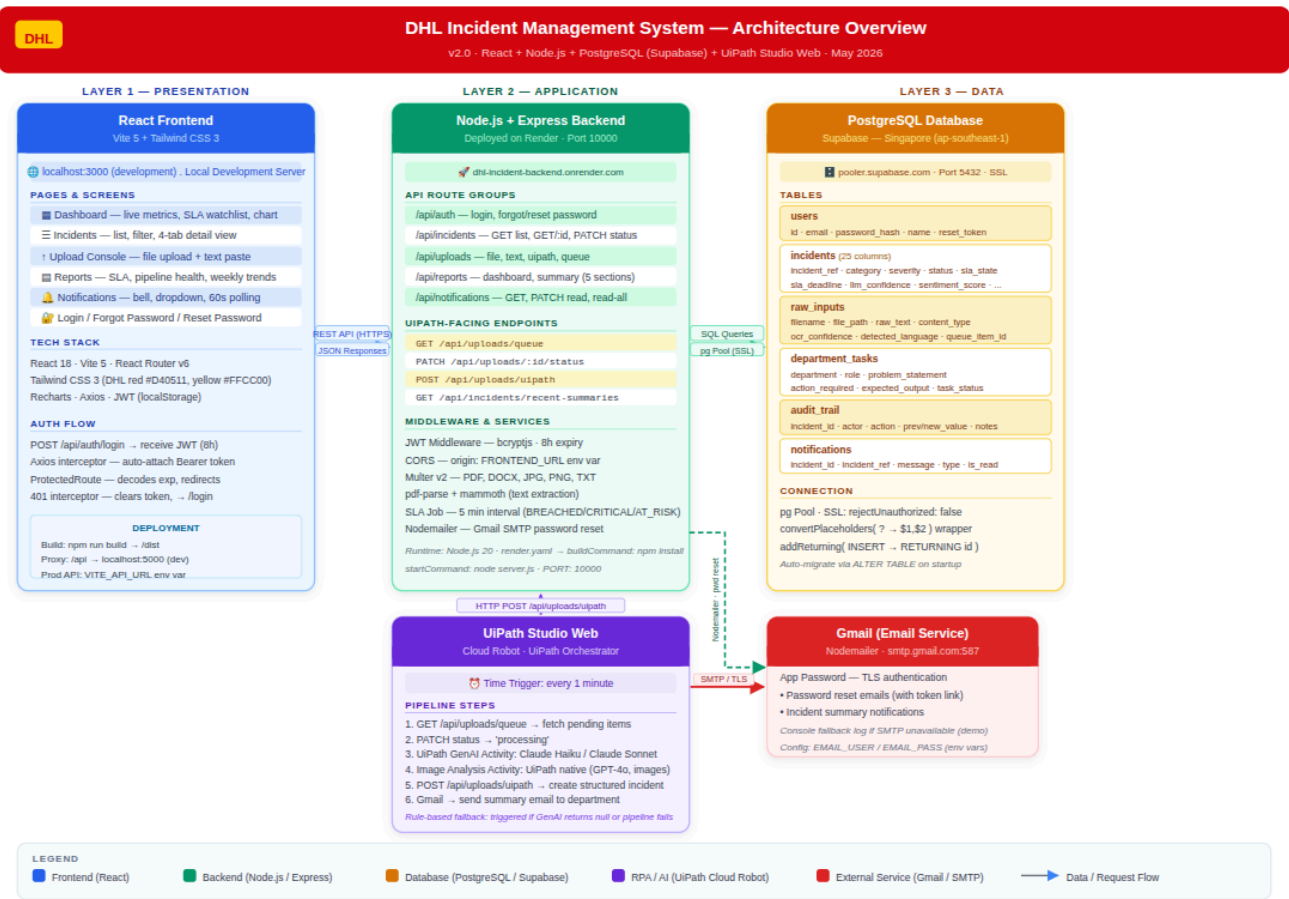


Figure 1 — DHL Incident Management System Architecture Overview

2.2 Layer 1 — React Frontend (Presentation Layer)

The frontend is a single-page application built with React 18, Vite 5, and Tailwind CSS 3, styled in DHL's official brand colours (red #D40511, yellow #FFCC00). It runs locally on port 3000 and communicates with the backend via Axios, with a JWT Bearer token auto-attached to every request. A 401 interceptor clears the token and redirects to login on any unauthorized response.

The application provides six interfaces: a **Dashboard** (live metrics, SLA watchlist, weekly chart); an **Incidents** page (filterable list, four-tab detail view); an **Upload Console** (PDF, DOCX, image, text); a **Reports** page (SLA performance, pipeline health, trend analysis); a **Notifications** panel (SLA alerts, 60-second polling); and an **Authentication** flow (login, forgot password, email-based reset).

2.3 Layer 2 — Node.js + Express Backend (Application Layer)

The backend is built with Node.js 20 and Express, deployed on Render at <https://dhl-incident-backend.onrender.com> (port 10000). It handles all business logic, API routing, file processing, SLA management, and email delivery across five route groups: /api/auth (authentication and password reset), /api/incidents (full incident lifecycle), /api/uploads (file ingestion and UiPath integration), /api/reports (dashboard metrics), and /api/notifications (SLA alerts).

File uploads are handled by Multer v2 (PDF, DOCX, JPG, PNG, TXT). Text extraction runs at upload time using pdf-parse and Mammoth — not at processing time — because Render's ephemeral filesystem cannot guarantee file persistence after the request completes. Extracting and storing raw text immediately ensures the RPA pipeline always has content to process when it polls the queue.

A background SLA job runs every 5 minutes, updating incident SLA states and inserting notification records on every state change. Password reset emails are delivered via Nodemailer over Gmail SMTP with TLS.

2.4 Layer 3 — PostgreSQL Database (Data Layer)

The database is hosted on Supabase and accessed via SSL connection pooling. A backend wrapper auto-converts ? placeholders to PostgreSQL \$1, \$2 syntax and appends RETURNING id to all INSERT statements. Schema changes are applied automatically via ALTER TABLE on server startup.

The database contains six tables:

Table	Purpose
users	Agent credentials and JWT reset tokens
incidents	Core incident record — 25 columns covering AI output, SLA data, and status
raw_inputs	Uploaded file metadata, extracted text, and queue state
department_tasks	Per-department task assignments per incident
audit_trail	Immutable log of every status change and actor
notifications	SLA-triggered alerts with read/unread state

The incidents table is the central hub — all five other tables link back to it via incident_id, forming a clean one-to-many structure, as shown in Figure 2.

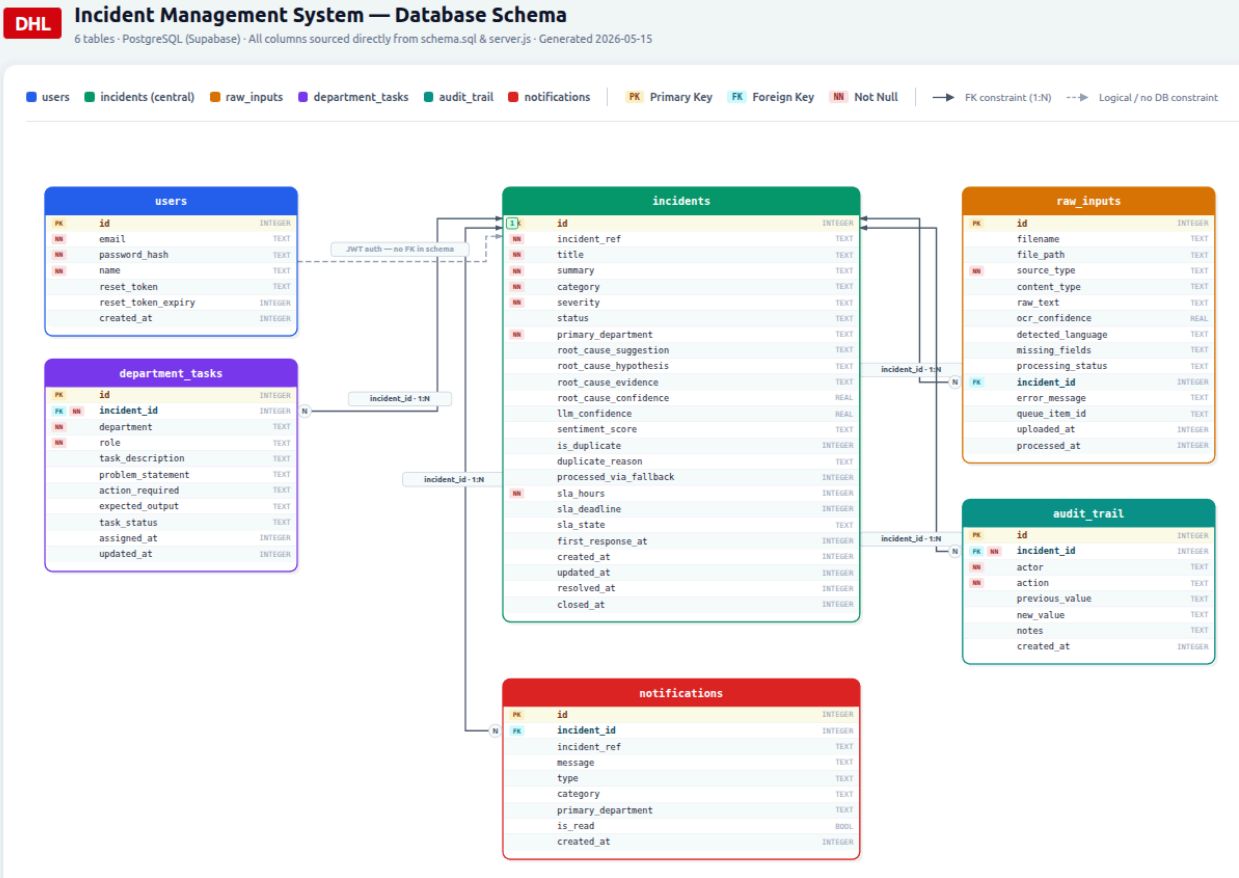


Figure 2 — Database Schema and Table Relationships

2.5 RPA Automation Layer — UiPath Studio Web

The RPA layer runs as a cloud robot on UiPath Orchestrator, triggered every 60 seconds. It polls the backend for pending records, processes them through the AI pipeline, posts structured incident cards back to the backend, and sends a Gmail summary email at the end of each batch. Because it runs as a cloud robot, it interacts with the system exclusively through the same REST API used by the frontend — fully decoupled from the web application's internal code. The full pipeline is documented in Section 5.

2.6 AI Intelligence Layer

Three models are used, each matched to its task:

1. **Claude Haiku** — all NLP tasks: language detection, named entity recognition (14 types), sentiment analysis, content rewriting, summarisation, duplicate detection, and categorisation (8 categories).
2. **Claude Sonnet** — content generation, synthesising all NLP outputs into a complete structured JSON incident card. Sonnet's stronger reasoning is reserved for this final step.
3. **GPT-4o** — image analysis via UiPath's native Image Analysis activity, processing photographs, screenshots, and handwritten notes.

All models are accessed through UiPath GenAI activities. Each activity is followed by a null check — if any returns empty, the fallbackActivated flag is set to True.

2.7 Fallback Architecture

The system is designed to function reliably even when AI services are unavailable. If fallbackActivated is True after any null response, or the content generation step returns empty JSON, the pipeline switches to rule-based fallback processing. It uses keyword matching for category, urgency patterns for severity, a routing table for department assignment, and text extraction for the summary. The resulting incident card is marked processed_via_fallback: true and llm_confidence: 0.0, displayed visibly to agents in the incident detail view.

UiPath Studio Web does not support Try/Catch or screenshot capture. The Boolean flag achieves equivalent resilience — any failure at any stage triggers the fallback and processing continues. No record is left unprocessed.

2.8 Incident Status State Machine

Every incident progresses through a defined six-state lifecycle that reflects its operational journey from intake to closure, as shown in Figure 3:

New → Assigned → In Progress → Pending → Resolved → Closed

A Cancel transition is available from any active state — New, Assigned, In Progress, and Pending. Incidents are never deleted from the system. Every transition, including cancellations, is recorded in the audit trail with the actor, timestamp, previous status, and new status, providing a complete and tamper-proof history of every incident's lifecycle. Once an incident reaches Closed, it is archived with no further transitions permitted.

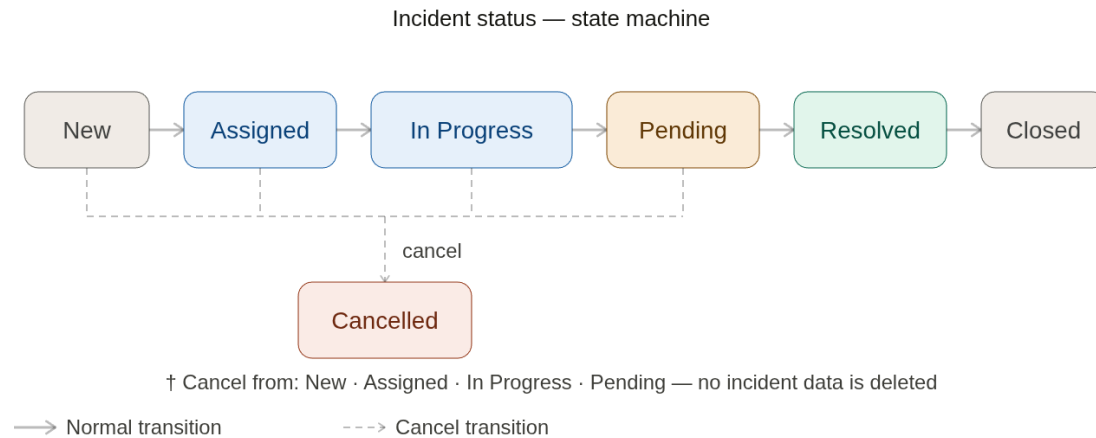


Figure 3 : Incident Status State Machine

2.9 SLA Framework

Each incident receives an SLA deadline on creation based on severity:

Severity	SLA Deadline
Critical	6 hours
High	24 hours
Medium	48 hours
Low	72 hours

SLA progresses through five states: **ON_TRACK** → **AT_RISK** (>50% elapsed) → **CRITICAL** (>80%) → **BREACHED** → **COMPLETED**. The backend evaluates all active incidents every 5 minutes and updates states accordingly. Every state change triggers a notification, visible in the frontend bell panel within 60 seconds. SLA state is displayed on every incident card and on the dashboard's Critical Watch List.

2.10 Key Architectural Decisions

The following decisions were made deliberately and each addresses a specific operational or technical constraint:

- **Time trigger over queue trigger** — A time-based poll every 60 seconds is simpler, more predictable, and easier to monitor than a message queue. It requires no additional infrastructure and is fully supported by UiPath Studio Web.
- **Text extraction at upload time** — Render's ephemeral filesystem cannot guarantee file persistence beyond the upload request. Extracting text immediately and storing it in the database ensures the RPA pipeline always has content to process, regardless of server restarts.
- **PostgreSQL over SQLite** — PostgreSQL supports concurrent connections, is production-grade, and is natively hosted on Supabase with automatic backups. SQLite was used in early development but was replaced to meet the requirements of a cloud-deployed, multi-user system.
- **Cloud robot over local robot** — UiPath Studio Web eliminates any dependency on a local machine being online. The automation runs reliably on UiPath's cloud infrastructure, accessible from anywhere.
- **Boolean flag over Try/Catch** — Studio Web does not support Try/Catch. The flag-based approach provides equivalent resilience: any failure propagates gracefully through the pipeline and triggers the fallback without crashing the execution.
- **Single Main.xaml workflow** — All pipeline logic is contained in one file, making debugging straightforward and execution context consistent across all processing steps.

SECTION 3 — API AND JSON STRUCTURE

3.1 Overview

The system exposes a RESTful API built with Node.js and Express, returning JSON responses across all endpoints. All routes except login and password reset are protected by JWT Bearer token authentication. The API serves two consumers — the React frontend for user-facing operations, and the UiPath RPA layer for automated incident processing.

3.2 Key API Endpoints

Method	Endpoint	Purpose	Consumer
POST	/api/auth/login	Login, returns JWT (8h)	Frontend
POST	/api/auth/forgot-password	Send password reset email	Frontend

GET	/api/incidents	List incidents with filters	Frontend
GET	/api/incidents/:id	Get single incident detail	Frontend
PATCH	/api/incidents/:id/status	Update incident status	Frontend
POST	/api/uploads	Upload file to queue	Frontend
GET	/api/uploads/queue	Poll pending records	UiPath
PATCH	/api/uploads/:id/status	Mark record as processing	UiPath
GET	/api/uploads/:id/content	Get extracted text	UiPath
GET	/api/uploads/:id/file	Get image as base64	UiPath
POST	/api/uploads/uipath	Submit structured incident card	UiPath
GET	/api/incidents/recent-summaries	Fetch summaries for duplicate check	UiPath
GET	/api/reports/dashboard	Dashboard metrics	Frontend
GET	/api/notifications	Get SLA notifications	Frontend
PATCH	/api/notifications/:id/read	Mark notification as read	Frontend

3.3 Incident Card JSON Structure

The core output of the AI pipeline is a structured incident card JSON posted by UiPath to POST /api/uploads/uipath. This JSON drives the entire incident creation process — the backend parses it, creates the incident record, assigns department tasks, and starts SLA tracking.

```
json{

  "raw_input_ids": [12],

  "title": "COD Amount Discrepancy — RM450 Collected vs RM250 Invoiced",

  "summary": "Customer reports courier collected RM450 COD on delivery...",

  "category": "COD Dispute",

  "severity": "Critical",

  "primary_department": "Finance",

  "supporting_departments": ["Warehouse", "Customer Support"],

  "root_cause_hypothesis": "COD sticker may have been handwritten incorrectly",

  "root_cause_evidence": "Customer photo of receipt shows RM450; system shows RM250",

  "root_cause_suggestion": "Verify COD sticker against dispatch manifest",

  "sentiment_score": "Negative",

  "detected_language": "English",

  "is_duplicate_likely": false,

  "duplicate_reason": null,

  "processed_via_fallback": false,

  "llm_confidence": 0.92,

  "department_tasks": [

    {

      "department": "Finance",

      "role": "COD Verification",

      "problem_statement": "RM200 discrepancy between collected and invoiced amount",
```

```
"action_required": "Cross-check COD collection record against dispatch manifest",  
  
"expected_output": "Confirmed correct amount and initiate refund if applicable"  
  
}  
  
]  
  
}
```

When the fallback activates, `processed_via_fallback` is set to true and `llm_confidence` to 0.0, clearly indicating to agents that the record was processed by rule-based logic rather than AI.

3.4 Error Handling

All API responses follow a consistent format. Successful responses return `{ data: ... }` or `{ message: "..." }` with a 200 or 201 status. Errors return `{ error: "..." }` with the appropriate HTTP status code (400 for validation, 401 for unauthorized, 404 for not found, 500 for server errors). The RPA layer detects failures by checking whether `postResponseBody` contains the string "error", incrementing the `countFailed` counter accordingly.

SECTION 4 — User Interface Screenshots

4.1 Login Page

As shown in Figure 4, JWT-secured authentication is styled in DHL's official brand colours. The Forgot Password link triggers a token-based email reset flow. Demo credentials are displayed for evaluation purposes.

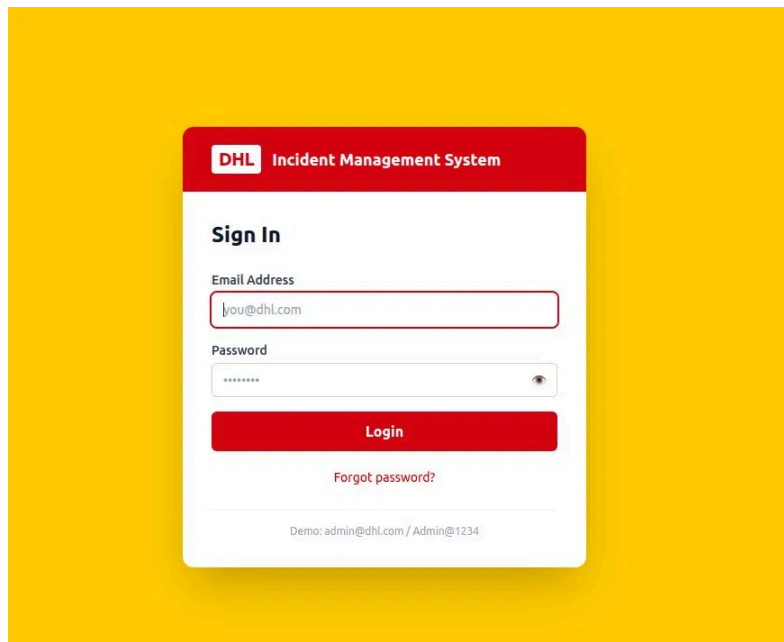


Figure 4 — Login Page

4.2 Dashboard Page

As shown in Figure 5, the operational command centre showed four live metric cards — Total Today, Pending, Resolved Today, and Overdue (highlighted red). The Critical Watch List surfaces all BREACHED incidents with their assigned department and SLA state. The Recent Activity feed shows real-time audit entries including UiPath-automated actions such as duplicate flagging and incident creation. The stacked bar chart breaks down incident volume by category across the current week.

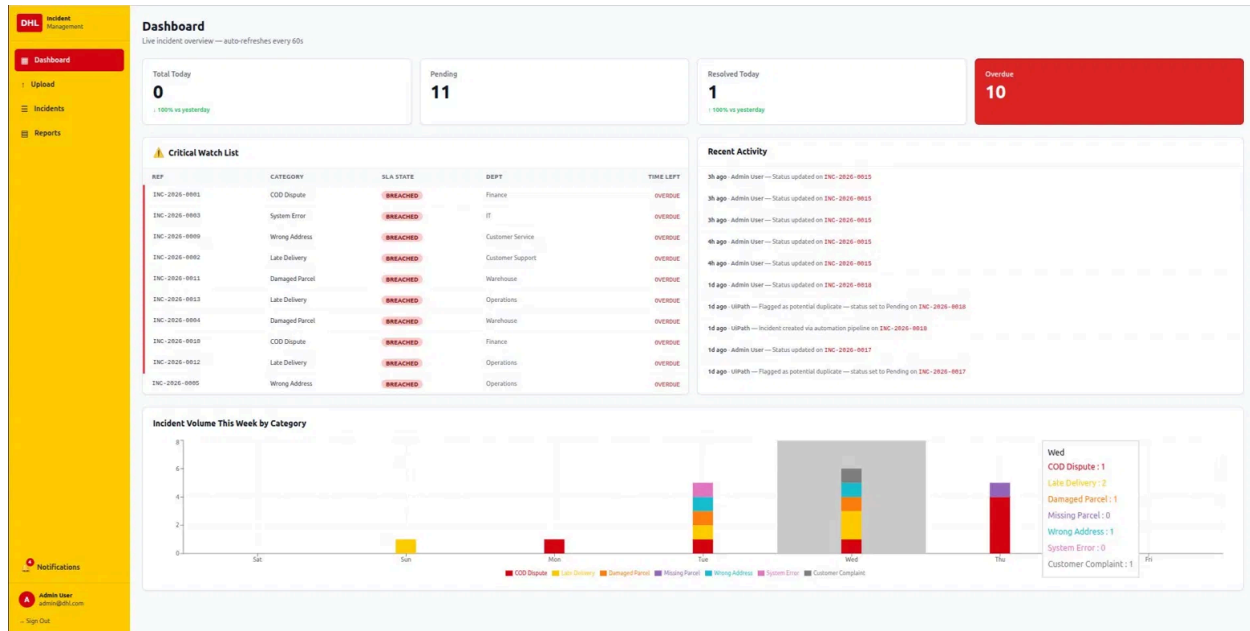


Figure 5 : Dashboard

4.3 Upload Page

As shown in Figure 6, agents can submit raw incident inputs via file upload (PDF, DOCX, JPG, PNG) or by pasting plain text directly. The Processing Queue on the right displays all submitted records with their file type, submission time, and processing status — refreshing every 10 seconds. All records shown are marked Processed, confirming successful RPA pipeline execution.

DHL Incident Management

Upload Raw Information
Submit files or text for WPath to process and classify

Upload File

Drop files here or click to browse
Supported: PDF, DOCX, JPG, PNG

— or —

Paste Raw Text
Phone notes, typed messages, email content

Paste incident text here...

Submit Text

Processing Queue
Automatically refreshes every 10 seconds

FILE / SOURCE	TYPE	TIME	STATUS
1778760333800_Customer_WhatsApp_m... Manual	Text	14 May, 08:05 pm	✓ Processed
1778757325887_Customer_WhatsApp_m... Manual	Text	14 May, 07:15 pm	✓ Processed
Text Input Text Input	Text	14 May, 05:37 pm	✓ Processed
1778733608171_Missing_Parcel_Incident... Manual	Pdf	14 May, 12:40 pm	✓ Processed
1778732295684_Customer_WhatsApp_m... Manual	Text	14 May, 12:18 pm	✓ Processed
1778639054793_Email_from_procureme... Manual	Docx	13 May, 03:57 pm	✓ Processed
1778658215422_text_warehouse_note.png Manual	Image	13 May, 03:43 pm	✓ Processed
1778657649672_Incoming_email_from_c... Manual	Docx	13 May, 03:34 pm	✓ Processed
17786555616530_Customer_WhatsApp_m... Manual	Pdf	13 May, 03:00 pm	✓ Processed
1778646936134_Customer_email_dated_... Manual	Text	13 May, 12:35 pm	✓ Processed
Text Input Text Input	Text	13 May, 09:05 am	✓ Processed
Text Input Text Input	Text	13 May, 06:05 am	✓ Processed
damaged_parcel_photo.jpg Manual	Image	13 May, 05:05 am	✓ Processed
damage_report_form.pdf Manual	Pdf	13 May, 05:05 am	✓ Processed

Notifications

Admin User
admin@dhl.com

Sign Out

Figure 6: Upload Console

4.4 Incident Management Page

As shown in Figure 7, the incidents list displays 18 records with colour-coded severity badges (Critical, High, Medium, Low), live SLA state indicators (BREACHED, CRITICAL, COMPLETED), and real-time status labels. Four dropdown filters — Status, Severity, Category, and SLA State — plus a free-text search bar allow agents to quickly locate specific incidents. Each row shows the assigned primary department, enabling fast triage without opening the incident.

DHL

Incident Management

Dashboard

Upload

Incidents

Reports

Notifications

Admin User
admin@dhl.com

Sign Out

Incident Management

18 total incidents

Search ref, title, summary...

All Statuses

All Severities

All Categories

All SLA States

Clear

REF	RECEIVED	CATEGORY	SEVERITY	STATUS	SLA STATE	DEPARTMENT	
INC-2026-0018	14 May 2026, 08:07 pm	COD Dispute	HIGH	Cancelled	COMPLETED	Finance	View ..
INC-2026-0017	14 May 2026, 07:17 pm	COD Dispute	HIGH	Cancelled	COMPLETED	Finance	View ..
INC-2026-0016	14 May 2026, 05:46 pm	COD Dispute	HIGH	Cancelled	COMPLETED	Finance	View ..
INC-2026-0015	14 May 2026, 12:41 pm	Missing Parcel	CRITICAL	Closed	COMPLETED	Operations	View ..
INC-2026-0014	14 May 2026, 12:19 pm	COD Dispute	HIGH	Closed	COMPLETED	Finance	View ..
INC-2026-0013	13 May 2026, 03:59 pm	Late Delivery	CRITICAL	New	BREACHED	Operations	View ..
INC-2026-0012	13 May 2026, 03:44 pm	Late Delivery	HIGH	New	BREACHED	Operations	View ..
INC-2026-0011	13 May 2026, 03:35 pm	Damaged Parcel	CRITICAL	New	BREACHED	Warehouse	View ..
INC-2026-0010	13 May 2026, 03:21 pm	COD Dispute	HIGH	New	BREACHED	Finance	View ..
INC-2026-0009	13 May 2026, 12:37 pm	Wrong Address	CRITICAL	New	BREACHED	Customer Service	View ..
INC-2026-0008	13 May 2026, 09:05 am	Customer Complaint	LOW	New	CRITICAL	Customer Support	View ..
INC-2026-0003	13 May 2026, 06:05 am	System Error	CRITICAL	New	BREACHED	IT	View ..
INC-2026-0004	13 May 2026, 05:05 am	Damaged Parcel	MEDIUM	In Progress	BREACHED	Warehouse	View ..
INC-2026-0005	13 May 2026, 01:05 am	Wrong Address	LOW	Pending	BREACHED	Operations	View ..
INC-2026-0002	12 May 2026, 07:05 pm	Late Delivery	HIGH	Assigned	BREACHED	Customer Support	View ..
INC-2026-0001	12 May 2026, 10:05 am	COD Dispute	CRITICAL	In Progress	BREACHED	Finance	View ..
INC-2026-0007	11 May 2026, 09:05 am	COD Dispute	MEDIUM	Closed	COMPLETED	Finance	View ..

Figure 7 : Incidents List

4.5 Incident Detail Page (Overview Tab)

As shown in Figure 8, the overview tab for e.g INC-2026-0015 shows the AI-generated summary with an LLM Confidence score of 88%, the Root Cause Analysis with hypothesis, supporting evidence, and a 78% confidence score, and the AI-assigned classification (Missing Parcel, Critical, Operations). The Status Management panel confirms the incident is Closed with no further transitions available. The AI Generated badge clearly distinguishes AI-produced content from manually entered data.

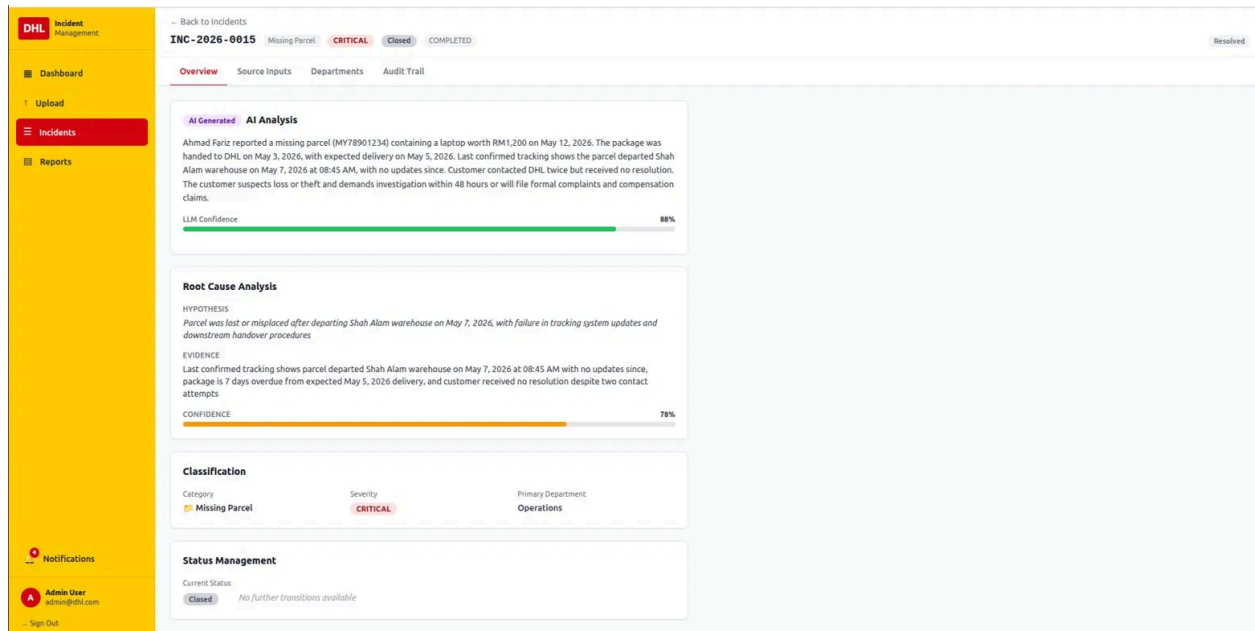


Figure 8 : Overview Tab

4.6 Incident Detail Page (Source Input)

As shown in Figure 9, the source inputs tab displays the raw file that the UiPath pipeline processed to create this incident — a PDF uploaded manually, with the detected language (English) labelled automatically. The extracted raw text is shown exactly as received from the customer, preserving the original unstructured input alongside the AI-generated analysis for full traceability.

The screenshot displays the 'Source Inputs' tab for incident INC-2026-0615. The interface includes a left sidebar with navigation options: Dashboard, Upload, Incidents (selected), and Reports. The top navigation bar shows the incident status as 'Missing Parcel', 'CRITICAL', 'Closed', and 'COMPLETED'. The main content area is titled 'Raw Fragments Merged Into This Incident' and shows '1 source(s) processed by UiPath GenAI pipeline'. A file named '1778733608171_Missing_Parcel_Incident.pdf' is listed, with a detected language of 'English'. The extracted text from the PDF is displayed, showing an email from Ahmad Fariz bin Zulkifli dated May 12, 2026, reporting a missing parcel with tracking number MY78901234. The text is preserved exactly as received, including line breaks and punctuation. A 'Show more...' link is visible at the bottom of the text block. The bottom of the page shows a user profile for 'Admin User' and a 'Sign Out' button.

Figure 9 : Source Inputs Tab

4.7 Incident Detail Page (Departments Tab)

As shown in Figure 10, the departments tab shows the AI-generated task assignments for each involved department. The primary owner (Operations) and three supporting departments (Warehouse, Customer Service, Finance) each receive a specific problem statement, action required, and expected output — generated automatically by the AI pipeline based on the incident category and context. Agents can update each department's task status independently.

The screenshot displays the 'Departments' tab for incident INC-2026-0015, categorized as 'Missing Parcel' and 'CRITICAL'. The interface includes a left sidebar with navigation options: Dashboard, Upload, Incidents (selected), and Reports. The top navigation bar shows 'Back to Incidents', the incident ID 'INC-2026-0015', its status 'Missing Parcel', and buttons for 'CRITICAL', 'Closed', and 'COMPLETED'. The 'Departments' tab is active, showing a list of departments and their assigned tasks.

Primary Owner: Operations

PROBLEM STATEMENT
Handle Missing Parcel incident.

ACTION REQUIRED
Investigate and resolve as primary owner.

EXPECTED OUTPUT
Resolution report and closure confirmation.

Buttons: Not Started, Mark In Progress

Supporting Departments

Warehouse (Status: Not Started)

PROBLEM
Parcel MY78901234 departed Shah Alam warehouse on May 7, 2026 at 08:45 AM with no subsequent tracking updates, indicating potential loss during transit or warehouse handling

ACTION REQUIRED
Conduct immediate physical search of Shah Alam warehouse, review CCTV footage, verify loading manifests, and trace courier handover records from May 7, 2026

EXPECTED OUTPUT
Detailed warehouse investigation report with parcel location status or confirmed loss declaration within 24 hours

Updated 1d ago

Customer Service (Status: Not Started)

PROBLEM
Customer contacted DHL twice without receiving resolution or satisfactory updates, escalating to formal complaint threats and compensation demands

ACTION REQUIRED
Initiate priority case escalation, provide hourly updates to customer, and prepare compensation processing documentation in accordance with DHL liability policy

EXPECTED OUTPUT
Customer communication log, formal apology letter, and compensation offer letter for RM1,200 laptop value

Updated 1d ago

Finance (Status: Not Started)

PROBLEM
Potential compensation claim of RM1,200 for lost high-value item requires financial approval and processing

ACTION REQUIRED
Review insurance coverage, validate parcel declared value, and prepare compensation payment authorization if loss is confirmed

EXPECTED OUTPUT
Compensation assessment report and approved payment authorization within 48 hours

Updated 1d ago

Footer: Notifications, Admin User (admin@dhl.com), Sign Out

Figure 10 : Departments Tab

4.8 Incident Detail Page (Audit Trail Tab)

As shown in Figure 11, the audit trail provides a complete tamper-proof history of INC-2026-0015. The first entry, logged by UiPath, confirms the incident was created via the automation pipeline with confidence 0.88 and fallback:false. Subsequent entries show each manual status transition made by Admin User, from New through to Closed, with actor and timestamp recorded for every step. The Export CSV button allows the full trail to be downloaded for compliance reporting.

The screenshot displays the 'Audit Trail' tab for incident INC-2026-0015. The interface includes a left sidebar with navigation options: Dashboard, Upload, Incidents (selected), and Reports. The top navigation bar shows 'Back to Incidents' and the incident details: INC-2026-0015, Missing Parcel, CRITICAL, Closed, and COMPLETED. The Audit Trail section lists the following entries:

Actor	Action	From Status	To Status	Timestamp
UiPath	Incident created via automation pipeline	-	-	14 May 2026, 12:47 pm
Admin User	Status updated	New	Assigned	15 May 2026, 05:45 pm
Admin User	Status updated	Assigned	In Progress	15 May 2026, 05:46 pm
Admin User	Status updated	In Progress	Pending	15 May 2026, 06:33 pm
Admin User	Status updated	Pending	Resolved	15 May 2026, 06:33 pm
Admin User	Status updated	Resolved	Closed	15 May 2026, 06:33 pm

An 'Export CSV' button is located at the top right of the Audit Trail section. The bottom of the page shows a user profile for 'Admin User' (admin@dhl.com) and a 'Sign Out' link.

Figure 11 : Audit Trail Tab

4.9 Report Page (Daily Operational Summary and SLA Performance)

As shown in Figure 12, the reports page auto-refreshes every 60 seconds and opens with four operational metrics — Total Today, Resolved Today, Pending, and Breached Active (highlighted red). The SLA Performance section breaks compliance down by severity tier, by department, and by average resolution time, giving management a clear view of where SLA failures are concentrated. The Critical Watch List below surfaces all actively breached incidents with department and severity visible at a glance.

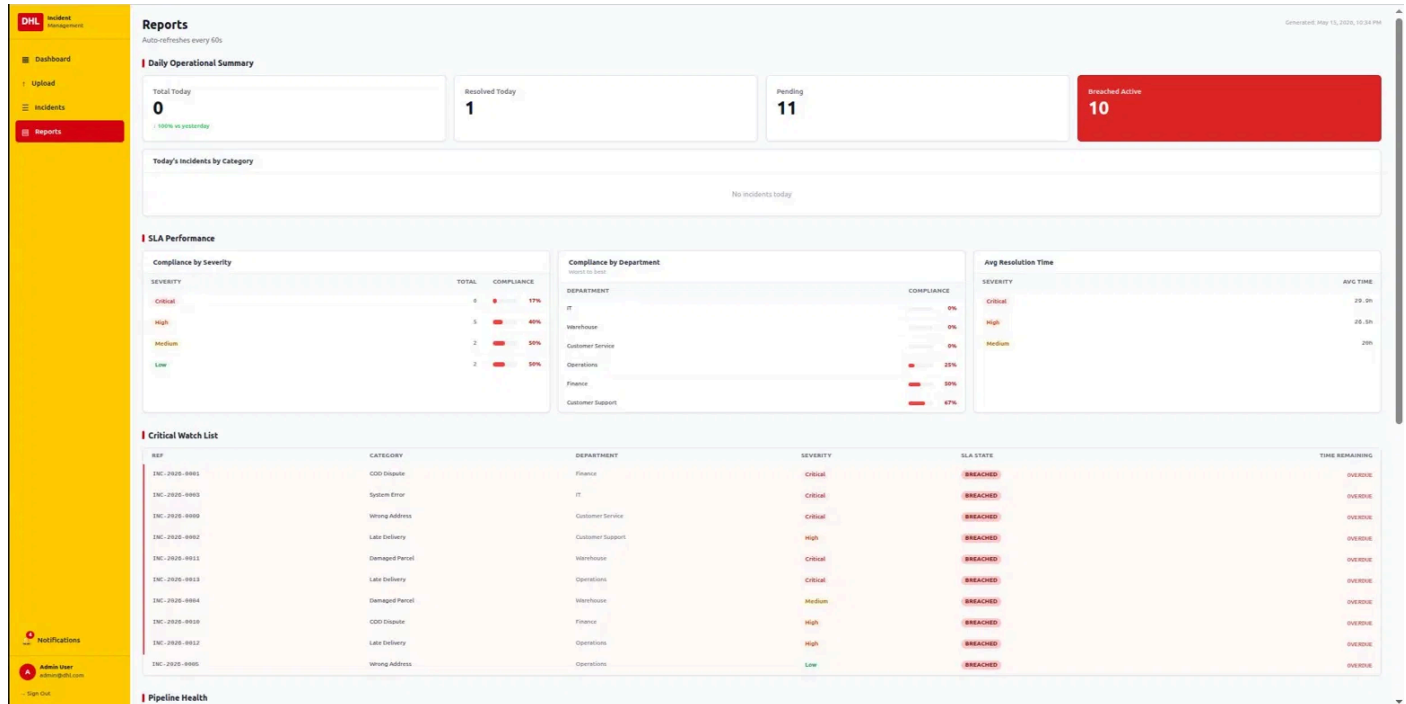


Figure 12 : Daily Operational Summary and SLA Performance

4.10 Report Page (Pipeline Health and Weekly Trend Intelligence)

As shown in Figure 13, The Pipeline Health section shows that all 20 processed incidents were handled by the GenAI pipeline — 100% success rate, 0% fallback, and an average LLM confidence of 86%. The Weekly Trend Intelligence section provides a stacked bar chart of incident volume by category, identifies the top three incident categories (COD Dispute, Late Delivery, Damaged Parcel), and flags the highest SLA breach rate department (IT at 100%). This section gives management actionable intelligence for resource allocation and process improvement.

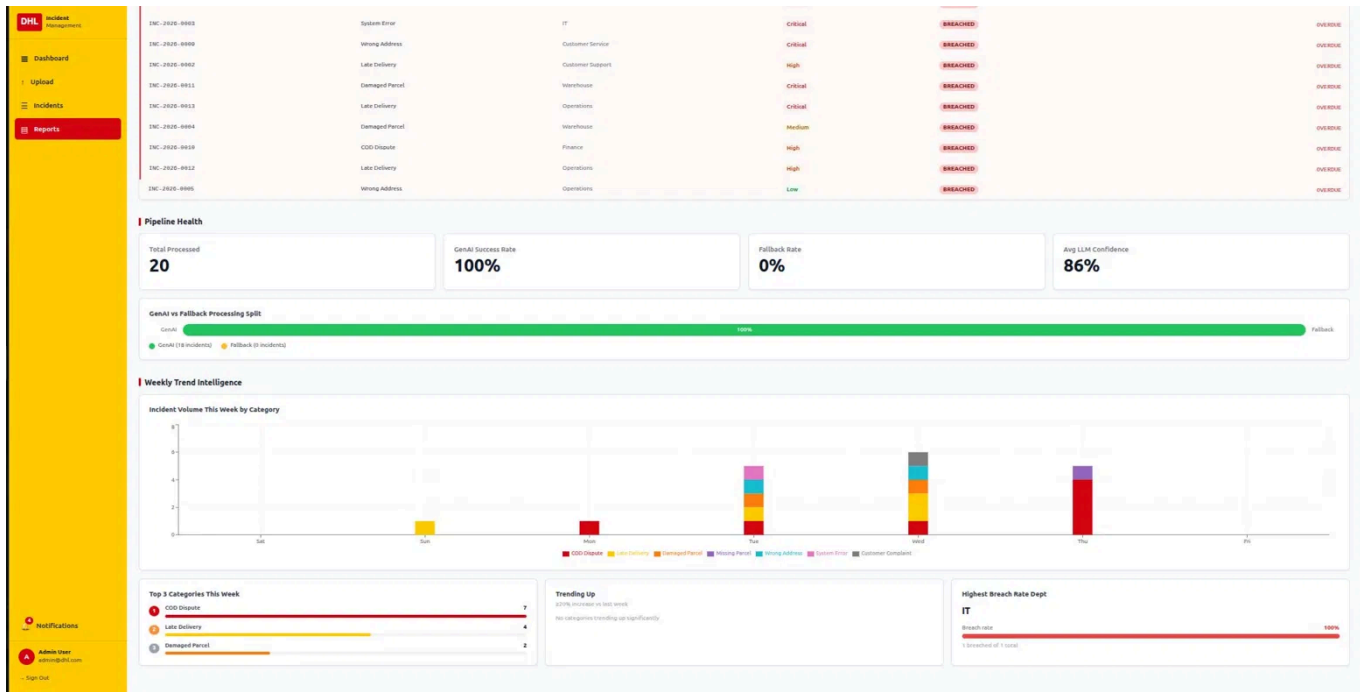


Figure 13 : Pipeline Health and Weekly Trend Intelligence

4.11 Notification Panel

As shown in Figure 14, the bell icon in the navigation bar shows a live unread count badge. The notifications dropdown lists SLA-triggered alerts in chronological order, each showing the incident reference, the specific SLA event (AT_RISK, CRITICAL, BREACHED), and the time remaining or elapsed. Unread notifications are marked with a red dot. A Mark all read button clears the panel. Notifications are generated automatically by the backend SLA job and appear in the panel within 60 seconds of the state change.

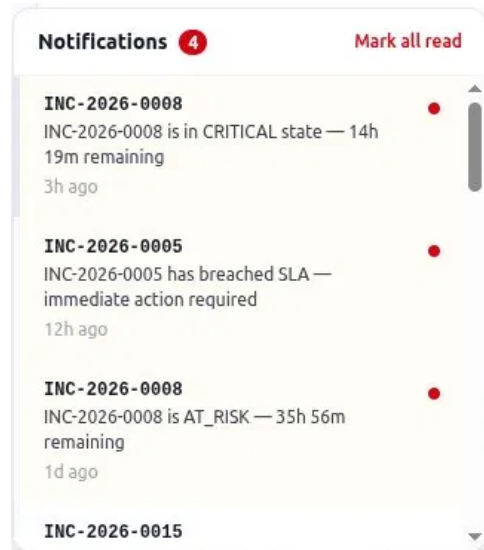
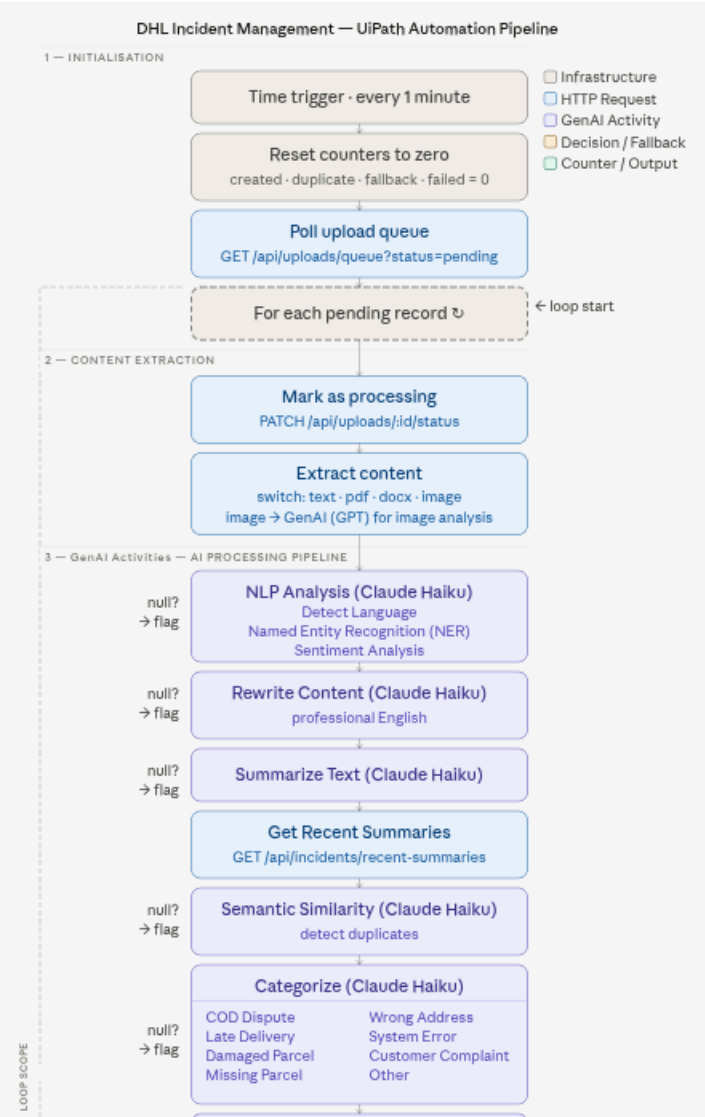


Figure 14 : Notifications Panel

SECTION 5 — RPA WORKFLOW AND AUTOMATION EXPLANATION

5.1 Overview

The UiPath Studio Web automation pipeline is the intelligence layer that transforms raw, unstructured inputs into clean, structured incident records — without any human involvement. The pipeline runs as a cloud robot triggered every 60 seconds, processes all pending records in a single batch, and posts the results directly to the backend. The full workflow is illustrated in Figure 15.



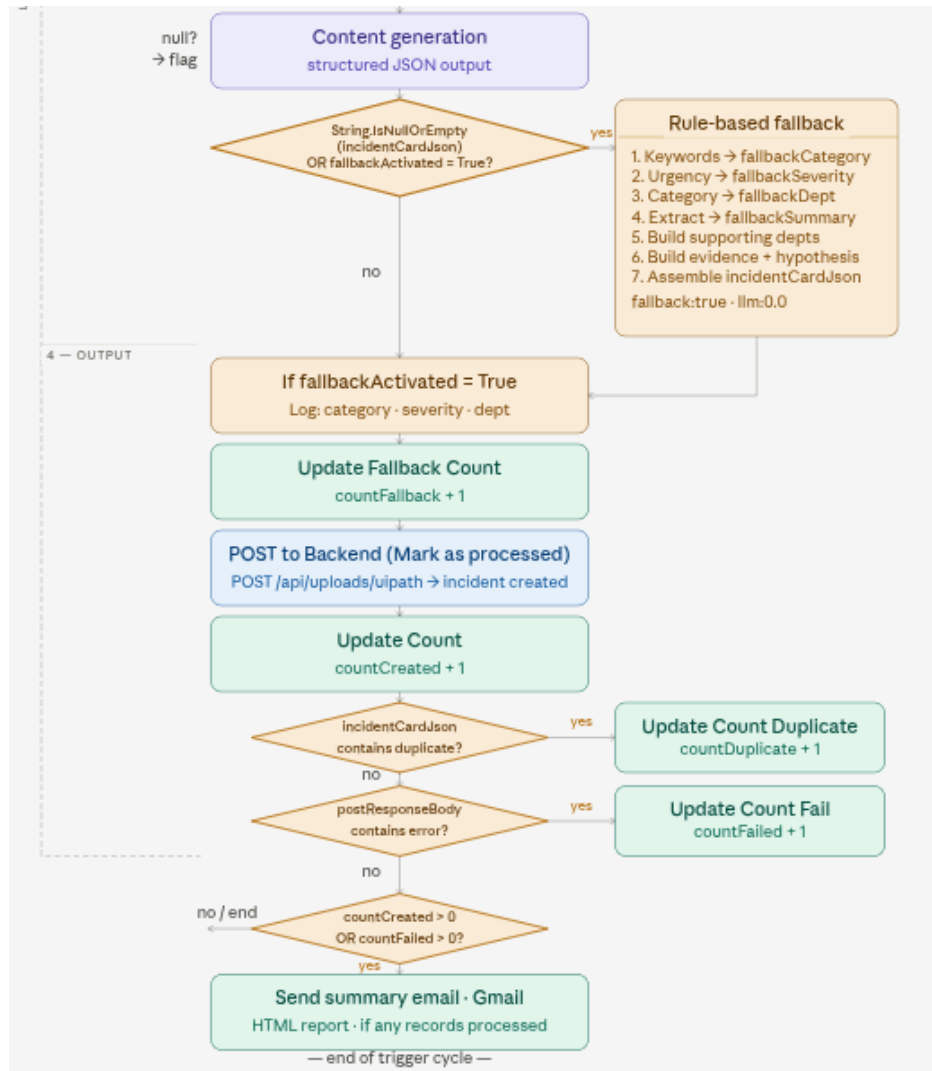


Figure 15 — UiPath Automation Pipeline

5.2 Stage 1 — Initialisation

At the start of every trigger cycle, four counters are reset to zero: created, duplicate, fallback, and failed. These track the outcome of every record processed in that cycle and are used at the end to decide whether to send a summary email.

The pipeline then polls the backend via GET /api/uploads/queue?status=pending to retrieve all records waiting to be processed. If records are found, a **For Each** loop begins — processing one record at a time until the queue is empty.

5.3 Stage 2 — Content Extraction

For each record, the pipeline immediately PATCHes its status to processing, preventing any duplicate processing if the trigger fires again before the current cycle completes.

It then extracts the text content based on the file type. Text, PDF, and DOCX files are handled via GET /api/uploads/:id/content, which returns the pre-extracted text stored at upload time. Image files (JPG, PNG) are fetched as base64 via GET /api/uploads/:id/file and passed to UiPath's native Image Analysis activity powered by GPT-4o, which reads photographs, screenshots, and handwritten notes and returns extracted text. All paths produce a single extractedText variable that flows into the AI pipeline.

5.4 Stage 3 — AI Processing Pipeline

The extracted text passes through seven sequential AI activities, all powered by Claude Haiku:

1. **NLP Analysis** — detects the language, extracts 14 named entity types (tracking numbers, amounts, dates, departments, locations, etc.), and determines the sentiment of the input.
2. **Rewrite Content** — rewrites the raw input into professional formal English, removing informal language, spelling errors, and channel-specific noise.
3. **Summarize Text** — produces a concise 100-word summary of the incident.
4. **Get Recent Summaries** — fetches recent incident summaries from GET /api/incidents/recent-summaries to provide context for the next step.
5. **Semantic Similarity** — compares the current input against recent summaries to detect potential duplicates. **Figure 16** demonstrates this in action — INC-2026-0018 was flagged with a similarity score of 0.62, triggering the Possible Duplicate Detected alert visible on the incident overview."
6. **Categorize** — assigns one of eight incident categories: COD Dispute, Late Delivery, Damaged Parcel, Missing Parcel, Wrong Address, System Error, Customer Complaint, or Other.
7. **Content Generation** — Claude Sonnet synthesises all prior outputs into a complete structured JSON incident card, including title, summary, root cause hypothesis, evidence, department tasks, severity, and classification.

Each activity is followed by a null check. If any activity returns an empty response — due to model unavailability, a rate limit, or an unprocessable input — the fallbackActivated flag is set to True and processing continues to the next step rather than stopping.

Duplicate Detection in Action (INC-2026-0018) Incident INC-2026-0018, a COD Dispute flagged as Cancelled with COMPLETED SLA, shows the AI-generated duplicate detection alert. A similarity score of 0.62 triggered the "Possible Duplicate Detected" warning, identifying that multiple related summaries exist with identical amounts, departments, and core facts. The AI Analysis achieved 88% LLM confidence and the Root Cause Analysis identified the hypothesis at 72% confidence — both generated automatically by the pipeline. This demonstrates the semantic similarity step functioning correctly in a real processing cycle.

The screenshot displays the DHL Incident Management system interface. On the left is a yellow sidebar with navigation links: Dashboard, Upload, Incidents (highlighted), Reports, Notifications, and a user profile for Admin User (admin@dhl.com) with a Sign Out option. The main content area shows the incident details for INC-2026-0018, categorized as a COD Dispute with a HIGH severity and CANCELLED status. The interface includes tabs for Overview, Source Inputs, Departments, and Audit Trail. The Overview tab is active, showing an AI-generated analysis. The AI Analysis section describes a discrepancy in a Cash on Delivery (COD) transaction with DHL, where the customer reported being charged RM450 instead of the RM250 on the invoice, leading to an alleged RM200 overcharge. It notes that the Finance Department records confirm the system shows RM450, contradicting the original invoice. A green progress bar indicates 88% LLM Confidence. Below this, the Root Cause Analysis section provides a hypothesis: 'Manual alteration or system data entry error occurred between invoice generation (RM250) and COD sticker preparation (RM450), with inadequate verification controls allowing the discrepancy to reach delivery stage'. It also lists evidence: 'Finance Department confirms system shows RM450 while original invoice states RM250; Warehouse documentation notes uncertainty about whether COD sticker was handwritten or printed; customer provided photographic evidence of RM450 receipt contradicting RM250 invoice'. An orange progress bar shows 72% Confidence. The Classification section shows the Category as COD Dispute, Severity as HIGH, and Primary Department as Finance. A yellow warning box states 'Possible Duplicate Detected' with a similarity score of 0.62, indicating multiple related summaries exist. The Status Management section shows the current status as Cancelled with the note 'No further transitions available'.

DHL Incident Management

Back to Incidents

INC-2026-0018 COD Dispute **HIGH** Cancelled COMPLETED

Overview Source Inputs Departments Audit Trail

AI Generated AI Analysis

A discrepancy has been identified in a Cash on Delivery (COD) transaction with DHL. The customer reported being charged RM450 upon delivery, while the invoice stated RM250, resulting in an alleged RM200 overcharge. The customer provided photographic evidence of the receipt. Finance Department records confirm the system shows RM450, contradicting the original invoice. Warehouse documentation notes uncertainty about whether the COD sticker amount was handwritten or printed. The customer has requested a RM200 refund. The source of the discrepancy and responsible party remain unidentified.

LLM Confidence 88%

Root Cause Analysis

HYPOTHESIS
Manual alteration or system data entry error occurred between invoice generation (RM250) and COD sticker preparation (RM450), with inadequate verification controls allowing the discrepancy to reach delivery stage

EVIDENCE
Finance Department confirms system shows RM450 while original invoice states RM250; Warehouse documentation notes uncertainty about whether COD sticker was handwritten or printed; customer provided photographic evidence of RM450 receipt contradicting RM250 invoice

CONFIDENCE 72%

Classification

Category: COD Dispute Severity: HIGH Primary Department: Finance

Possible Duplicate Detected
Similarity score of 0.62 indicates multiple related summaries exist in comparison text addressing the same COD discrepancy incident with identical amounts, departments, and core facts

Status Management

Current Status: Cancelled No further transitions available

Figure 16: Duplicate Detection Flagged

5.5 Stage 4 — Fallback Decision

After content generation, the pipeline evaluates a single decision condition: **String.IsNullOrEmpty(incidentCardJson) OR fallbackActivated = True?**

This condition checks two things simultaneously — whether the content generation step produced an empty JSON output, and whether the fallbackActivated flag was raised at any earlier stage in the pipeline. Either condition alone is sufficient to trigger the fallback. If No — the AI pipeline successfully produced a valid, complete incident card JSON. The workflow proceeds directly to the output stage without any fallback processing.

If Yes — the rule-based fallback activates immediately and executes the following seven steps in sequence:

1. **Keyword matching → fallbackCategory:** The extracted text is scanned for predefined keywords associated with each of the eight incident categories. For example, keywords such as "COD", "cash", "amount", and "overcharge" map to the COD Dispute. The first matching category is assigned as fallbackCategory.
2. **Urgency keyword detection → fallbackSeverity:** The text is scanned for urgency indicators such as "urgent", "immediately", "critical", and "escalate" to determine severity. Strong urgency keywords assign High severity; the absence of urgency indicators defaults to Medium or Low.
3. **Category-to-department routing → fallbackDept:** A static routing table maps each category to its responsible primary department. For example, COD Dispute routes to Finance, Damaged Parcel routes to Warehouse, and System Error routes to IT. This table mirrors the same routing logic used by the AI pipeline.
4. **Text extraction → fallbackSummary:** The first 400 characters of the extracted text are taken as the fallbackSummary. The text is sanitised to remove special characters and excessive whitespace before being stored.
5. **Build supporting departments:** Based on the assigned category, a predefined list of supporting departments is constructed. For example, a COD Dispute incident involves Finance as primary, with Warehouse and Customer Support as supporting departments.
6. **Build evidence and hypothesis:** Key sentences are extracted from the raw text to form the fallbackEvidence. A standard hypothesis statement is constructed based on the category, providing agents with a starting point for investigation even without AI reasoning.

7. **Assemble incidentCardJson:** All fallback variables are assembled into a complete, valid incidentCardJson — structurally identical to the AI-generated output. The card is explicitly marked with `processed_via_fallback: true` and `llm_confidence: 0.0`, which the backend stores and displays visibly in the incident detail view. This makes it immediately clear to agents that the record was processed by rule-based logic rather than AI, prompting them to review and enrich the details manually if needed.

This fallback mechanism ensures that no record is ever left unprocessed. Even in the complete absence of AI availability, the system continues to function, routing every incident to the correct department with the correct severity — maintaining operational continuity at all times.

5.5.1 Fallback Logic Validation

To validate this mechanism, the fallback was triggered under controlled test conditions by disabling the Content Generation activity, causing it to return an empty JSON output. This satisfied the `String.IsNullOrEmpty(incidentCardJson)` condition at the decision gateway, activating the rule-based fallback. Figures 17 and 18 demonstrate the result.

As shown in Figure 17, a controlled test incident demonstrating the rule-based fallback mechanism. The amber banner "Processed via rule-based fallback — LLM was unavailable. Manual review recommended" confirms the fallback activated successfully. The system correctly identified the category as Wrong Address and severity as Medium through keyword matching. The Root Cause confidence score of 30% distinguishes rule-based output from AI-generated analysis, signalling to agents that manual review is recommended.

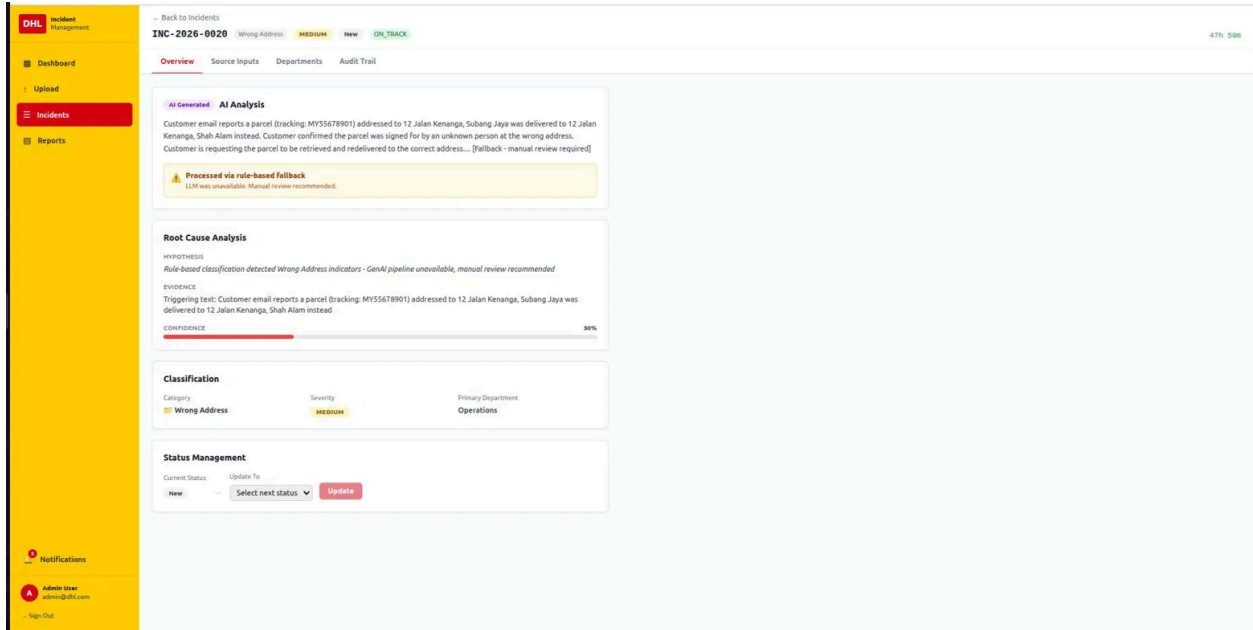


Figure 17: Fallback Incident Overview (INC-2026-0020)

As demonstrated in Figure 18, the audit trail confirms the incident was created by UiPath with {"fallback":true} recorded at the point of creation on 16 May 2026 at 10:30 AM — providing a tamper-proof record that the fallback mechanism executed and was logged correctly.

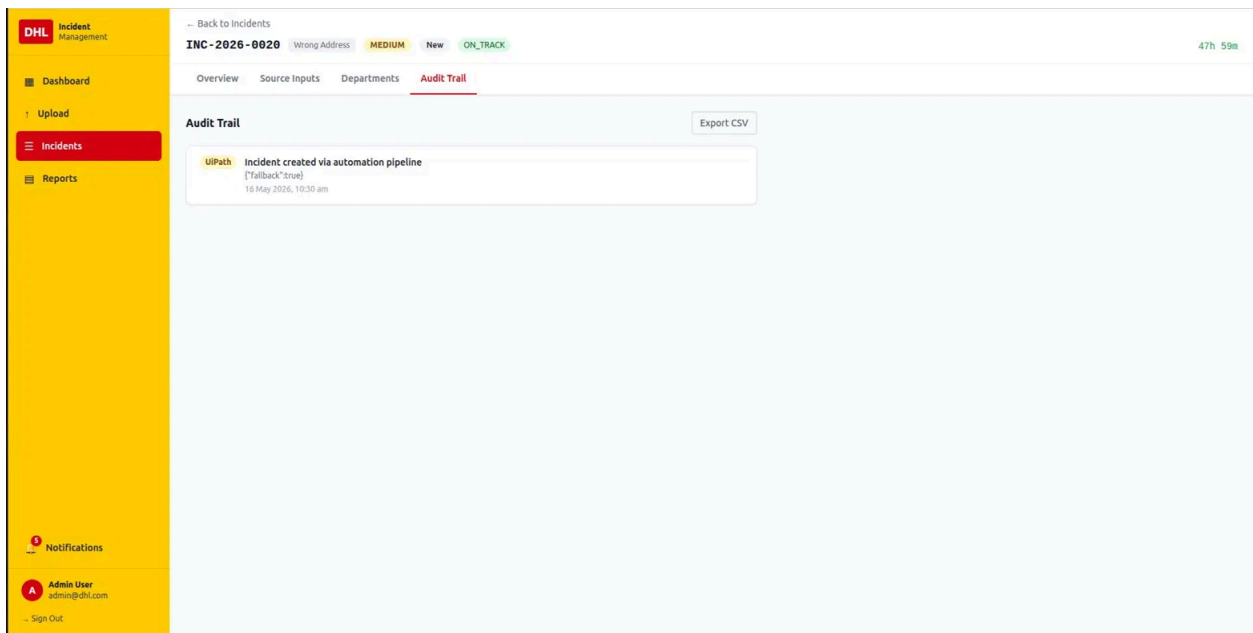


Figure 18: Fallback Audit Trail (INC-2026-0020)

5.6 Stage 5 — Output and Summary Email

After the fallback decision, all paths converge at the output stage:

- If `fallbackActivated = True`, the category, severity, and department are logged and `countFallback` is incremented by 1.
- The incident card is posted to `POST /api/uploads/uipath`, which creates the incident in the database, assigns department tasks, and starts the SLA timer. `countCreated` increments by 1.
- If the incident card contains `is_duplicate_likely: true`, `countDuplicate` increments by 1.
- If the POST response contains "error", `countFailed` increments by 1.

Once the loop has processed all pending records, the pipeline evaluates one final condition outside the loop: **`countCreated > 0 OR countFailed > 0`**. If true, a Gmail summary email is sent as an HTML report listing all four counter values — providing the operations team with a full batch summary after every processing cycle.

As UiPath Studio Web cloud robots operate without persistent local filesystem access, a log file cannot be attached directly to the email; the counter summary serves as the equivalent operational record of each batch run.

As shown in Fig 19, an automated HTML summary email sent by the UiPath pipeline after each processing cycle, styled with DHL's official branding. The email reports four counters for the batch run on 14 May 2026 at 12:07 PM — 1 incident created, 1 duplicate detected, 0 processed via fallback, and 0 failures — confirming the pipeline executed successfully end-to-end and the AI-based duplicate detection triggered correctly.

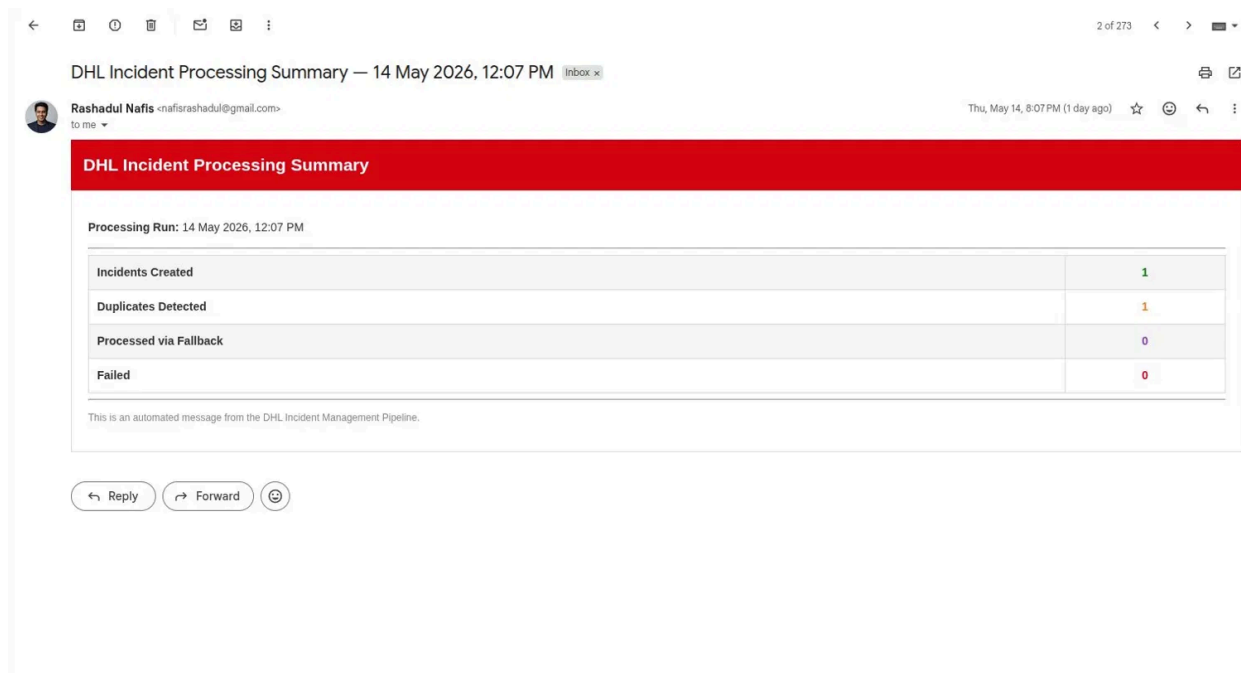


Figure 19 — UiPath Pipeline Summary Email (Gmail)